Effect of Tool-Count Context on MCP Tool Selection

MCPToolBench++ subset, 5 vs 20 vs 50 tools in context. AI465 Agentic AI, SVNIT Surat, Semester VII. Research Project Phase 2 report. Roll no. U23AI052 (individual work).

1. Objective and declared deviations

Approved objective (Phase 1). Evaluate the effect of tool-count context on MCP tool-selection accuracy and failure composition for single-step and multi-step tool-calling tasks on a 200-task stratified subset of MCPToolBench++ (search, map, finance), at a matched tool-count budget (5 vs 20 vs 50 tools in context), measured by AST accuracy, AST DAG accuracy, tokens per task, model calls per task and wall-clock latency per task. It targets three gaps in the paper (Fan et al., 2025, arXiv:2508.07575): no cost or latency, no variance, no failure taxonomy.

#	Deviation from the approved objective	Reason
D1	Subset is 200 **single-step** tasks (67 search, 67 map, 66 finance); no multi-step tasks, so AST DAG accuracy is not reported.	The public repository (commit 2bd35e5) ships single-step data for these domains; map has only 2 multi-step items out of 500. Scope reduction, reported as allowed.
D2	No execution-based Pass@K.	It needs live MCP servers with third-party credentials. It was not in the approved metric list.
D3	The benchmark's GPT-4o judge is replaced by a deterministic scorer (`rp2/scoring.py`).	Removes judge/model circularity and cost. Checked by an oracle test (the 200 gold labels score 100%), the unit-test suite, and agreement with the author's own labels on 20 tasks is measured by `scripts/label_audit.py` (not yet done).
D4	20- and 50-tool contexts add **distractors from a pool of 87 tools** taken from all six benchmark categories.	A task's native list is 1 (finance), 5 (search) or 32 (map) tools. Gold tool plus at most 2 labelled similar tools are always present; contexts are nested (5 is a subset of 20 is a subset of 50).
D5	"Matched budget" is implemented as (a) equal prompts across conditions and (b) a 5-tool baseline given k=8 samples with majority vote so its token spend is close to the 50-tool condition.	Rubric item 2.4: a method that costs more must be compared against a baseline given the same budget.
D6	A missing required parameter counts as a parameter error.	Stricter than the benchmark judge, which only checks parameters that were predicted.

2. What was implemented

- **Data** (`scripts/build_subset.py`): seeded stratified sample from the benchmark clone, saved to `data/subset_200.json` with the 87-tool pool; the checkout is self-contained.
- **Condition builder** (`rp2/context.py`): the only thing that differs between conditions is the tool list. One seeded permutation of the pool per task gives the nested design; order is a seeded shuffle.
- **Prompts** are files (`prompts/`), identical across conditions; a test asserts this. The padding arm appends neutral text of exactly the length difference between that task's 50- and 5-tool contexts.
- **Runner** (`rp2/runner.py`): provider adapters for Anthropic and OpenAI-compatible endpoints, retries with backoff, global rate limit, resume, jobs interleaved across conditions within each run so model drift hits every condition equally. A failed call becomes an `api_error` record, never a crash.
- **Scorer and taxonomy**: tool_ok (gold or a labelled similar tool), param_ok (gold values compared with numeric, enum, lat/lon (0.1) and token-F1 ≥ 0.5 rules, as in the judge prompt), and nine mutually exclusive failure labels.
- **Analysis**: mean ± SD over runs, paired bootstrap over tasks (10,000 resamples), tokens per correct task, domain and position breakdowns, and an automatically selected traced failure case.
- **Ablations** (each isolates one factor): position of the gold tool (first / last / shuffled at 50 tools); context length (5 tools padded to the 50-tool length); budget (5 tools with self-consistency).
- **Own vs borrowed.** About 2,655 lines of Python written for this project (runner, conditions, scorer, statistics, report, slides, tests). Borrowed: the benchmark's task data and labels (github.com/mcp-tool-bench/mcptoolbenchpp, commit 2bd35e5) and the parameter-matching rules from its judge prompt, re-implemented. No benchmark code is imported.

3. Experimental setup (pinned)

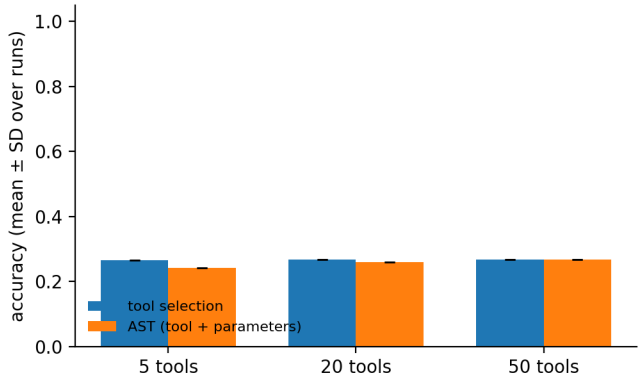
Model requested	gemini-3.5-flash-lite	Model string returned by API	gemini-3.5-flash-lite (x492)
Provider	openai_compat	Dates of runs (UTC)	2026-10-14T13:21:09Z to 2026-10-14T15:05:20Z
Temperature / top-p	0.7 / 1.0	Seed	not supported by provider
Runs per condition	1 full passes over 200 tasks	max_tokens	512
Data hash (subset)	90dbba2ad0077423	Repo commit	5515b2c1a8c5

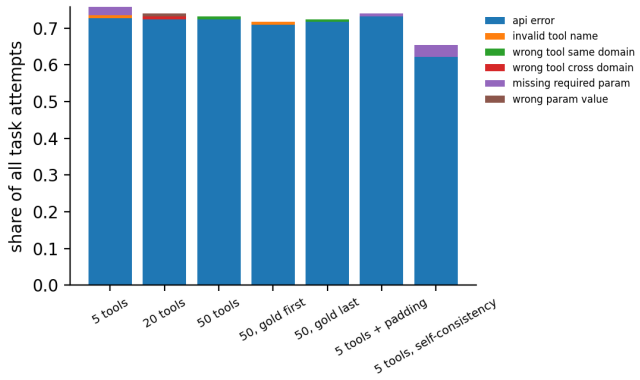
Task metric: AST accuracy (tool and parameters correct) and tool-selection accuracy. Cost: model calls, prompt and completion tokens (provider-reported) and wall-clock latency per task. Every run writes a full trace (prompts, tool list, raw response, usage, score) to `traces/`; `python scripts/aggregate.py` regenerates every table from them. Planned cost (chars/4 estimate, input tokens): about 8.2M for the three core conditions and 29.8M with all ablations over 3 runs.

4. Results

Condition	Tool acc %	AST acc %	Param acc % | tool	In tok / task	Calls / task	Latency s	Tok / correct
5 tools	26.6	24.2	91.2	505	1.0	13.83	2115
20 tools	26.8	26.0	97.1	1228	1.0	13.96	4757
50 tools	26.8	26.8	100.0	2707	1.0	14.11	10143
50, gold first	28.3	28.3	100.0	2855	1.0	14.08	10103
50, gold last	27.6	27.6	100.0	2761	1.0	13.94	10049
5 tools + padding	26.8	26.0	97.1	1990	1.0	13.95	7690
5 tools, self-consistency	37.8	34.6	91.7	4039	8.0	112.15	11838

Mean ± SD over 1 runs of the 200-task subset. SD is across runs; uncertainty across tasks is in the paired bootstrap intervals below.





5. Analysis

- AST accuracy over 5 / 20 / 50 tools: 24.2% / 26.0% / 26.8% (run-to-run SD n/a / n/a / n/a pp); tool-selection accuracy 26.6% / 26.8% / 26.8%.
- 5 to 50 tools changes AST accuracy by +2.4 pp, 95% CI [-4.7, +10.2] (paired bootstrap over 127 tasks): not distinguishable from task-level sampling noise.
- Step 5 to 20: +1.6 pp, 95% CI [-4.7, +7.9]. Step 20 to 50: +0.8 pp, 95% CI [-3.9, +5.5].
- Cost: mean input tokens per task 505 / 1228 / 2707 (x5.4 from 5 to 50 tools); tokens per correct task 2115 / 4757 / 10143.
- Latency per task 13.83s / 13.96s / 14.11s; model calls per task 1.00 in every core condition.
- Failure composition: tool-selection errors are 1% of failures at 5 tools and 1% at 50. Most common failure: api error (96%) at 5 tools, api error (99%) at 50.
- Position control, 50 tools with gold tool first vs shuffled: +1.6 pp, 95% CI [-3.9, +7.1].
- Position control, 50 tools with gold tool last vs shuffled: +0.8 pp, 95% CI [-5.5, +7.1].
- Length control, 5 tools padded with neutral text to the 50-tool length vs plain 5 tools: +1.6 pp, 95% CI [-6.3, +9.4].
- Matched budget: 5 tools with self-consistency (k=8) uses 4101 tokens per task against 2716 for 50 tools (ratio 1.51); AST accuracy 34.6% vs 26.8%.
- Self-consistency (5 tools) to 50 tools at similar spend: -7.9 pp, 95% CI [-14.2, -1.6].
- AST accuracy by domain, 5 to 50 tools: search 46% to 51%; map 0% to 0%.

Reading of the evidence (rule-based from the numbers above; verify against the traces):

- No effect of tool count was detected at this sample size; with 200 tasks only large effects are visible, so this is evidence of a small effect, not of none.
- Position of the correct tool does not explain the 50-tool result (both position arms are within noise of the shuffled order).

Traced failure case (task 27c74bb0, search; solved in every 5-tool run, failed in every 50-tool run).
Query: *Can you extract content from the Careers section of the website www.company.com in text format?*. Gold tool: `tavily-crawl`, gold arguments `{"url": "https://www.company.com", "categories": ["Careers"], "format": "text"}`.
50-tool run 1: called `tavily-extract` with `{"format": "text", "categories": ["Careers"], "extract_depth": "basic", "urls": ["https://www.company.com"]}`; failure label **wrong_tool_same_domain**, gold tool at position 15 of 50. picked tavily-extract, expected ['tavily-crawl']
Description of the tool it picked: *A powerful web content extraction tool that retrieves and processes raw content from specified URLs, ideal for data collection, content analysis, and research tasks.*
Description of the gold tool: *A powerful web crawler that initiates a structured web crawl starting from a specified base URL. The crawler expands from that point like a tree, following internal links...*
Cause (write after reading the trace): _____

6. Limitations and four more weeks

- One model and one temperature: conclusions are about this model on this date, not about tool count in general. Single-step tasks only; no Pass@K or live MCP execution.
- Labels are the benchmark's. A "wrong tool in the same domain" may be a valid alternative that the labelled `similar_tools` list misses, which inflates same-domain errors; the traced case and the 20-task audit are the guard.
- Distractors come from other domains and are mostly easy to dismiss, so the 50-tool condition is gentler than a marketplace of near-duplicate tools. Parameter matching is lenient (containment and token F1), as in the benchmark. The public benchmark may be in the model's training data.
- **With four more weeks:** several models and sizes; distractors chosen by embedding similarity to the gold tool; a retrieval baseline that shows the top-k tools; multi-step chains built by composing the labelled calls; live MCP execution for Pass@K; failure labels audited by a second person.